Linear stability analysis in 'py-pde'

David Zwicker
(Dated: February 22, 2025)

CONTENTS

I. General derivation 1
A. Linearization first 1
B. Fourier transform first 1
C. Rule-of-thumb 1

II. Concrete examples 1
1. Compound functions 1
A. Compositions 2

I. GENERAL DERIVATION

We consider dynamical equations of the form

$$\partial_t c = f(c, \nabla c, \nabla^2 c, \dots) \,, \tag{1}$$

which we want to analyze around a homogeneous stationary state, $c(\boldsymbol{r}, t) = c_0 + \varepsilon \delta c(\boldsymbol{r}, t)$. Here, c_0 is chosen such that it is a stationary state, $f(c_0, 0, \dots) = 0$. To do this analysis, we need to linearize and move to Fourier space. Below, we check whether the order of operations matters.

A. Linearization first

Developing the equation up to first order in ε, we thus find

$$\partial_t \delta c = \sum_{n=0}^{\infty} \left.\frac{\partial f}{\partial(\nabla^n c)}\right|_{c_0} \cdot \nabla^n \delta c \,. \tag{2}$$

Note that the term containing derivatives of f is a scalar, vector, or tensor independent of position $\boldsymbol{r}$, which is contracted with (derivatives of) the field δc. Because δc enters linearly, we impose $\delta c(\boldsymbol{r}, t) = \exp(\omega t + i\boldsymbol{q}.\boldsymbol{r})$, effectively moving to Fourier space,

$$\omega(\boldsymbol{q}) = \sum_{n=0}^{\infty} \left.\frac{\partial f}{\partial(\nabla^n c)}\right|_{c_0} \cdot (i\boldsymbol{q})^n \,, \tag{3}$$

which should be the equation we want.

B. Fourier transform first

Since the form discussed above requires derivatives of the right hand side with respect to the derivatives of fields, which is difficult to implement in `sympy`, we here check whether we can first replace the derivatives by going to Fourier space and then linearize. We thus first consider the Fourier transformed function using $c(\boldsymbol{r}, t) = \tilde{c}(\boldsymbol{q}, t) e^{i\boldsymbol{r}\boldsymbol{q}}$,

$$\partial_t \tilde{c} = \tilde{f}(\tilde{c}, i\boldsymbol{q}\tilde{c}, -\boldsymbol{q}^2\tilde{c}, \dots) = \tilde{g}(\tilde{c}, \boldsymbol{q}) \,. \tag{4}$$

We then expand the field up to first order in ε using $\tilde{c}(\boldsymbol{q}, t) = c_0 + \varepsilon \delta\tilde{c}(\boldsymbol{q}) e^{\omega t}$,

$$\begin{aligned} \omega(\boldsymbol{q}) &= \left.\frac{\partial \tilde{g}}{\partial \tilde{c}}\right|_{c_0} \\ &= \sum_{n=0}^{\infty} \left.\frac{\partial \tilde{f}}{\partial((i\boldsymbol{q})^n \tilde{c})}\right|_{c_0} \cdot (i\boldsymbol{q})^n \,, \end{aligned} \tag{5}$$

which is identical to Eq. (3).

C. Rule-of-thumb

What about simply replacing every ∇ by $i\boldsymbol{q}$? Does this work for functions where the differential operators are not directly applied to c?

II. CONCRETE EXAMPLES

1. Compound functions

To see whether Eq. (3) and Eq. (5) agree, we test a simple example,

$$\begin{aligned} f &= \nabla\big[A(c)\nabla B(c)\big] \\ &= (\nabla A(c)).(\nabla B(c)) + A(c)\nabla^2 B(c) \\ &= A'(c)B'(c)(\nabla c)^2 + A(c)B''(c)(\nabla c)^2 + A(c)B'(c)\nabla^2 c \,. \end{aligned} \tag{6}$$

First, Eq. (3) can be brought to the form where the terms are derivatives are directly applied to c.

$$\begin{aligned} \omega(\boldsymbol{q}) &= \sum_{n=0}^{\infty} \left.\frac{\partial f}{\partial(\nabla^n c)}\right|_{c_0} \cdot (i\boldsymbol{q})^n \\ &= -A(c_0)B'(c_0)\boldsymbol{q}^2 \,. \end{aligned} \tag{7}$$

Using the rule-of-thumb, we obtain $\omega(\boldsymbol{q}) = i\boldsymbol{q}\big[A(c) i\boldsymbol{q} B(c)\big]$, which gives the right scaling with $\boldsymbol{q}$, but does not lead to any derivatives in the functions A and B. TODO: Apparently, we need to linearize in the field first and then perform the Fourier transform. We

thus need to tell `sympy` how to linearize the differential operators This would imply

$$\begin{aligned} f &= \nabla\big[A(c_0+\delta c)\nabla B(c_0+\delta c)\big] \\ &\approx \nabla\big[\big(A(c_0)+A'(c_0)\delta c\big)\nabla\big(B(c_0)+B'(c_0)\delta c\big)\big] \\ &\overset{\text{nabla const}}{\approx} \nabla\big[\big(A(c_0)+A'(c_0)\delta c\big)B'(c_0)\nabla\delta c\big] \\ &\overset{\text{linearize}}{\approx} \nabla\big[A(c_0)\nabla B'(c_0)\delta c\big] \\ &\overset{\text{collect}}{\approx} A(c_0)B'(c_0)\nabla^2\delta c \\ &\overset{\text{Fourier}}{\approx} A(c_0)B'(c_0)(-\boldsymbol{q}^2\delta c) \end{aligned} \tag{8}$$

This works well in this simple case, but we somehow need to tell sympy how to simplify the expressions.

A. Compositions

For the simple case

$$f = A(\nabla^2 c) \tag{9}$$

we find

$$f = A(c_0 + \nabla^2\delta c)\ . \tag{10}$$

Assuming $\nabla^2\delta c$ is also small, we have

$$f \approx A(c_0) + A'(c_0)\nabla^2\delta c \tag{11}$$

For the more complicated case

$$f = B\big(\nabla^2\big[A(\nabla^2 c)\big]\big) \tag{12}$$

we find

$$\begin{aligned} f &= B\big(\nabla^2\big[A(c_0+\nabla^2\delta c)\big]\big) \\ &\approx B\big(\nabla^2\big[A(c_0)+A'(c_0)\nabla^2\delta c\big]\big) \\ &= B\big(A'(c_0)\nabla^4\delta c\big) \\ &\approx B'(0)A'(c_0)\nabla^4\delta c \end{aligned} \tag{13}$$

TODO: Can we simply replace all fields by $c_0 + \epsilon\delta c$ and use sympy to expand in ϵ? I guess we need to tell sympy that the differential operators are linear, but this might be possible. We might need to do this from the inside out, ensuring that each leaf-expression is linear. We can do this for each linear operator (expand inside). During this linearization we might be able to replace the differential operators by their Fourier transform